Personal Learning Journey Report

IT3091 Machine Learning — Individual Assessment Deliverable

Student ID: IT24102913

Group ID: 2026-AI-36

Project Title: ChefCast AI — Restaurant Revenue & Product Demand Forecasting

Assigned Notebooks: NB04 (Lens 1 — Revenue Validation, Stacking & Model Selection) and NB06 (Lens 2 — Product Demand Data Preparation & Feature Engineering)

Submission Date: October 2026

Section 1: My Technical Role & Specific Contributions

NB04 — Lens 1: Revenue Validation, Stacking & Model Selection

My responsibility in NB04 was the most methodologically critical task in Lens 1: implementing the formal evaluation framework, building the time-series-safe stacking ensemble, selecting the Lens 1 champion, and retraining it for production deployment. If NB03 is where ML models are built, NB04 is where they are rigorously tested, and the winner is selected on defensible, principled grounds.

Specific work I performed:

1. Evaluation Framework Design & Implementation:

I implemented the four-metric evaluation suite (WAPE, MAE, RMSE, ***R^2***) for both validation and test splits:

```
def evaluate_model(y_true, y_pred, split_name): mae = np.mean(np.abs(y_true - y_pred)) rmse = np.sqrt(np.mean((y_true - y_pred)**2)) wape = np.sum(np.abs(y_true - y_pred)) / np.sum(y_true) * 100 # Safe MAPE implementation mask = y_true > 0 mape = np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100 r2 = 1 - np.sum((y_true - y_pred)**2) / np.sum((y_true - np.mean(y_true))**2) return {"MAE": round(mae, 2), "RMSE": round(rmse, 2), "WAPE_percent": round(wape, 2), "MAPE_percent": round(mape, 2), "R2_Score": round(r2, 4)}
```

I explicitly implemented MAPE with a zero-mask (`y_true > 0`) to prevent division by zero on zero-revenue dates, while documenting that WAPE is the primary selection metric since it avoids this pathology entirely.

2. Whole-Date Chronological Split Enforcement:

I enforced the strict chronological split across the Lens 1 feature matrix:

- Training: 2024-03-27 to 2025-12-18 (632 days)
- Validation: 2025-12-19 to 2026-04-28 (131 days) — used exclusively for champion selection
- Test: 2026-04-29 to 2026-09-07 (132 days) — sealed until final evaluation

I implemented the split by filtering on the `Date` column directly rather than using integer indices, ensuring that calendar-date integrity is preserved regardless of any row ordering operations.

3. TimeSeriesStackingRegressor Implementation:

I built the custom stacking class (working with IT24102664 who implemented a parallel version for Lens 2). My Lens 1 implementation:

- **Base models:** Ridge Regression, Random Forest, LightGBM (three structurally diverse learners).
- **Meta-learner:** Ridge Regression (L2, ***\alpha=5.0*** — slightly lighter regularisation than the base Ridge since meta-features are already compressed OOF predictions).
- **Cross-validation:** `TimeSeriesSplit(n_splits=3)` generating three expanding window folds: fold 1 trains on first 33% and validates on next 33%; fold 2 trains on first 67% and validates on final 33%.

The key technical insight: the meta-learner Ridge is trained only on out-of-fold (OOF) predictions — it never sees any prediction made by a base model on data that base model was also trained on. This prevents the double-use contamination that standard stacking implementations suffer from when applied to time-series.

4. Formal Champion Selection:

After evaluating all five models on the Validation set, I ranked them by Validation WAPE and committed to the champion:

Rank	Model	Validation WAPE

■ 1	**LightGBM**	**30.33%** → Selected Champion
■ 2	Ridge Regression	30.54%
■ 3	Stacking Ensemble	31.32%
4	XGBoost	31.67%
5	Random Forest	32.17%
—	Baseline Seasonal Naive	46.22%

I documented that the test set was sealed at this point and that the champion was committed irrevocably before test evaluation. This discipline prevents test-set selection bias.

5. Unbiased Test Evaluation:

After committing to LightGBM as champion, I evaluated it on the held-out test set exactly once: Test WAPE = **24.99%**, Test MAE = **9,109.36 LKR**, Test ***R^2*** = **0.3521**. The fact that test WAPE (24.99%) is materially lower than validation WAPE (30.33%) indicates the model generalized well — the validation period (Dec 2025–Apr 2026) contained more volatile festive calendar dates (Christmas, New Year, Sinhala New Year) that inflated validation error, while the test period (May–Sep 2026) was a more stable mid-year window.

6. Production Champion Retraining:

I retrained the LightGBM champion on the combined Train + Validation dataset (extending the training window from December 2025 through April 2026) and serialized it as `lens1_champion_model.joblib`. This extended-window model is what powers the live forward forecast in the Streamlit dashboard.

7. Feature Importance Analysis:

I extracted LightGBM's Gini-based feature importance and documented the top-5 predictors:

Rank	Feature	Importance	Domain Meaning
1	`Revenue_lag_1`	6.55%	Yesterday's revenue momentum
2	`Revenue_lag_3`	6.34%	3-day recent trend
3	`Revenue_roll_std_7`	5.69%	Weekly volatility index
4	`Revenue_lag_14`	5.26%	Fortnightly payday pattern
5	`Revenue_lag_21`	5.21%	3-week seasonal anchor

The top-2 features being short lags (***{lag}_1, {lag}_3***) rather than the weekly lag (***{lag}_7***) surprised me — I expected the weekly lag to dominate. Investigating further, I found this reflects LightGBM's leaf-wise splitting: the algorithm discovers that recent-day momentum (lag_1/lag_3) captures regime changes (sudden festival surges, weather-driven drops) that the weekly pattern misses. The 7-day lag handles the baseline weekly cycle but misses day-to-day volatility; short lags complement it.

NB06 — Lens 2: Product Demand Data Preparation & Feature Engineering

My responsibility in NB06 was to build the Lens 2 feature engineering pipeline — constructing the 9,614-row long-format panel feature matrix from the aggregated demand series.

Specific work I performed:

1. Long-Format Panel Construction:

I transformed the wide-format product demand table (895 days × 11 items) into a long-format panel suitable for a shared panel regression model:

```
# Melt wide format to long panel demand_long = demand_wide.melt( id_vars=['Date'], value_vars=ITEM_NAMES, var_name='ItemName', value_name='Qty' ) # Sort by date then item for consistent temporal ordering demand_long = demand_long.sort_values(['Date', 'ItemName']).reset_index(drop=True)
```

2. Per-Item Lag & Rolling Feature Engineering:

The key challenge of Lens 2 feature engineering is that lag features must be computed per-item — not across the entire panel. `LUNCH`'s `lag_7` must reference LUNCH's demand 7 days ago, not BREAKFAST's demand 7 days ago.

I implemented this via pandas `groupby`:

```
for item in ITEM_NAMES: item_mask = demand_long['ItemName'] == item item_series = demand_long.loc[item_mask, 'Qty'].copy() # Per-item lag features (shift=1 applied per item) shifted = item_series.shift(1) demand_long.loc[item_mask, 'Qty_lag_1'] = shifted.values demand_long.loc[item_mask, 'Qty_lag_7'] = item_series.shift(7).values # Per-item rolling statistics (on shifted series) demand_long.loc[item_mask, 'Qty_roll_mean_7'] = shifted.rolling(7).mean().values demand_long.loc[item_mask, 'Qty_roll_std_7'] = shifted.rolling(7).std().values demand_long.loc[item_mask, 'Qty_roll_mean_30'] = shifted.rolling(30).mean().values
```

The per-item groupby structure is essential: computing a rolling mean across the full long-format panel without groupby would compute rolling means across item boundaries, creating nonsensical "averages" of LUNCH demand and BREAKFAST demand.

3. One-Hot Encoding for Product Entities:

I implemented 11 binary product indicator columns (`item_LUNCH`, `item_BREAKFAST`, etc.) using `pd.get_dummies()` with manual column ordering to ensure consistent feature column positions across training and inference:

```
ohe_cols = pd.get_dummies(demand_long['ItemName'], prefix='item', dtype=float) # Reorder to guarantee consistent column ordering ohe_cols = ohe_cols.reindex( columns=['item_' + item.replace(' ', '_') for item in sorted(ITEM_NAMES)], fill_value=0.0 ) demand_long = pd.concat([demand_long, ohe_cols], axis=1)
```

Consistent column ordering is critical for production inference: the model serialized at training time expects features in the exact same column sequence at prediction time.

4. Panel Leakage Audit:

I performed a cross-product leakage audit to verify that per-item lag features do not accidentally reference other items' demand. I confirmed by checking: for WATALAPPAN on a randomly selected date, the `Qty_lag_7` value equals WATALAPPAN's own demand exactly 7 days prior — not any other dish's demand on that date.

5. Final Matrix Validation:

Output: 9,614 item-days × 41 features (30 shared temporal + 11 item OHE), with 231 rows dropped due to 21-day lag burn-in at the start of each product's time series.

Section 2: Critical Challenges I Overcame

Challenge 1: Diagnosing and Choosing WAPE Over MAPE

When I initially implemented MAPE in the evaluation suite, I immediately encountered the issue: several days in the Validation set had zero revenue for specific items (the WATALAPPAN zero-demand days). The MAPE formula requires dividing by y_t, and division by zero produced `inf` or `NaN` values that propagated through the metric calculation, producing meaningless results.

I explored three solutions: (i) exclude zero-demand days from the MAPE calculation, (ii) add a small epsilon to the denominator, or (iii) switch to WAPE entirely. I rejected (i) because it would systematically favor models that predict non-zero on days when zero was actually the correct answer. I rejected (ii) because adding $\varepsilon = 1.0$ to a denominator of 0.0 produces a MAPE of 100% for a prediction of 1.0 — which is technically correct but obscures the aggregate error pattern. WAPE (option iii) eliminates this pathology entirely by summing all errors and dividing by total demand volume. I implemented and reported MAPE alongside WAPE but used WAPE exclusively for champion selection.

Challenge 2: Cross-Item Lag Feature Contamination

My first implementation of the Lens 2 lag features applied pandas `shift(1)` and `rolling(7).mean()` directly to the full 9,614-row panel without groupby — incorrectly computing rolling averages across item boundaries. The `lag_7` for the first BUTTER CAKE row of each day referenced the previous 7th row of the full panel, which was a completely different item's demand from a different date.

I detected this bug by manually inspecting the `Qty_lag_7` values for three consecutive WATALAPPAN rows and noticing they did not match WATALAPPAN's actual historical demand from 7 days prior. The fix required restructuring the entire feature computation inside a per-item `groupby` loop, which significantly increased implementation complexity but was essential for correctness.

Challenge 3: The Surprising Validation-vs-Test WAPE Gap for Lens 1

When the test WAPE for LightGBM (24.99%) was significantly lower than its validation WAPE (30.33%), I was initially concerned that perhaps the test set evaluation was incorrect, or that some validation contamination had occurred. After carefully re-checking the split dates and confirming no date overlap, I investigated the calendar composition of each split:

The Validation split (Dec 2025–Apr 2026) contains the festive calendar cluster of Christmas week, New Year, and Sinhala New Year — all of which create demand spikes significantly above the model's rolling-window-trained expectations, inflating the validation error. The Test split (May–Sep 2026) is a more stable mid-year window with fewer extreme calendar events. This explained the WAPE gap as a calendar composition artifact rather than test set leakage — an important distinction for honest reporting.

Section 3: Individual Learning Reflection (Fink's Taxonomy)

Foundational Knowledge

The deepest foundational learning from my NB04 and NB06 work is a thorough understanding of what "unbiased evaluation" means in ML and why it requires the specific triple-split architecture we implemented. I can now explain from first principles:

- Why random cross-validation is invalid for sequential time-series data.
- Why selecting champions based on test performance creates selection bias.
- Why the validation set must be strictly separated from both training and testing.
- What the mathematical consequence of temporal data leakage is (artificially optimistic validation metrics that do not generalise to deployment).

This conceptual framework is transferable to any sequential ML problem — financial time-series, medical monitoring, weather prediction — regardless of domain.

Application

The most practical skill I developed was building production-grade ML evaluation pipelines from scratch. My evaluation function handles edge cases (zero-demand MAPE safeguard), computes multiple metrics simultaneously, stores results in structured dictionaries, and produces formatted comparison tables — all requirements of a production ML reporting system, not just an academic exercise.

The per-item panel feature engineering using `groupby` taught me the specific implementation pattern required for multi-entity time-series panel data — a pattern that applies directly to financial panel data (per-stock rolling features), medical longitudinal data (per-patient lag features), and demand forecasting systems at scale.

Integration

NB04 required integrating three knowledge domains that are rarely taught together:

- **Statistical evaluation theory:** Understanding when and why to use WAPE vs. MAPE vs. RMSE, and the mathematical properties of each.
- **Time-series ML methodology:** Understanding the chronological split architecture and why temporal order matters.
- **Software engineering:** Implementing clean, maintainable, production-grade Python code that serializes model artifacts, stores evaluation results in JSON, and integrates with the Streamlit dashboard.

The integration of all three was only possible because the project forced me to actually build a working end-to-end system rather than completing disconnected theoretical exercises.

Critical Thinking

The most demanding critical thinking challenge was interpreting the feature importance results from NB04. When `Revenue_lag_1` and `Revenue_lag_3` ranked above `Revenue_lag_7` in LightGBM importance, my initial reaction was: "This contradicts the EDA finding that weekly cyclicality is the dominant pattern — something must be wrong." Rather than accepting or rejecting this at face value, I systematically investigated by computing the correlation between each lag feature and the revenue target variable independently, then comparing these correlations to the LightGBM importance values.

The finding: `Revenue_lag_7` has the highest Pearson correlation with revenue (0.71), yet `Revenue_lag_1` and `Revenue_lag_3` have higher LightGBM importance. This apparent contradiction resolves when you understand that LightGBM's Gini importance measures marginal split gain — how much additional predictive power a feature adds *given that other features are already in the model*. Since `Revenue_lag_7` is highly correlated with `Revenue_lag_14` and `Revenue_lag_21` (which also capture weekly patterns), its marginal contribution is reduced once those other weekly features are present. `Revenue_lag_1`, by contrast, provides momentum information that the weekly lags cannot substitute — making it more marginal valuable. This is the distinction between correlation and mutual information in a multivariate model context.

Human Dimension

My NB04 contribution produced the numbers that directly determine the dashboard's operational claims. When the Streamlit dashboard displays "Expected Revenue: LKR 41,500 ± 6,200" for next Monday, those numbers trace directly to the LightGBM model I trained, the test metrics I computed, and the 90th-percentile residual I calculated for the planning buffer. The specificity of the operational recommendation — not just "revenue will be around 40,000" but "between 35,300 and 47,700 with 90% historical buffer coverage" — is a direct consequence of my quantitative evaluation framework.

When I realised that a restaurant owner might make a supplier payment decision based on these numbers, the responsibility of accurate model evaluation became very real.

Caring

I developed a genuine commitment to reporting model limitations honestly and prominently. The test ***R^2*** for Lens 1 (0.3521) means 64.8% of daily revenue variance is unexplained. In a traditional academic setting, this might be considered a "poor" model. But in the context of a restaurant with genuine unpredictable footfall driven by weather, social media, and Sri Lankan cultural events, a model explaining 35% of a historically noisy variable — while reducing decision error by 42% relative to heuristic baselines — is a meaningful operational improvement. I documented this distinction clearly: the correct benchmark is not a theoretical perfect model but the real-world heuristic the restaurant currently uses.

Learning How to Learn

Implementing the entire evaluation pipeline from scratch — including the stacking ensemble, the evaluation suite, the champion selection process, and the production retraining step — taught me how to reason through a sequence of interconnected technical decisions where the order of operations matters and each decision constrains all subsequent decisions. The most transferable meta-learning was: *always reason about what you know at each step in the pipeline, and ensure that computations only use information that would be available at that step in a real production system*.

Section 4: Key Technical Skills Acquired

Skill	Demonstration
Rigorous model evaluation design	4-metric evaluation suite with temporal split enforcement
Custom stacking ensemble	TimeSeriesStackingRegressor with OOF meta-feature generation
Champion selection discipline	Documented validation-only selection; test evaluated exactly once
Per-entity panel feature engineering	Groupby-based lag and rolling features with cross-item contamination audit
Model artifact management	joblib serialization with consistent feature column ordering
Feature importance interpretation	Resolved apparent conflict between correlation and Gini importance

---

Section 5: What I Would Do Differently

1. **Implement a formal calibration check:** After selecting the LightGBM champion, I would implement a calibration diagnostic — plotting predicted vs. actual residuals across quantiles to verify that the ±15% planning band genuinely covers 85% of actual outcomes.
2. **Add grouped cross-validation for panel data:** For the Lens 2 inner stacking CV, I would experiment with GroupKFold splitting on calendar date groups rather than row-level TimeSeriesSplit, to ensure that all 11 product rows from a single date are always in the same fold.
3. **Build an automated model registry:** Rather than manually tracking model files and their performance metrics in separate files, I would implement an MLflow tracking server that logs all experiment parameters, metrics, and artifacts automatically — enabling reproducible model selection without manual spreadsheet tracking.

I certify that this Personal Learning Journey report reflects my genuine individual contribution, learning experiences, and reflections. All described technical work was performed by me in the notebooks and code assigned to my responsibility.

Student ID: IT24102913

Date: October 2026